

Проектирование конвейера данных и выбор хранилищ

Платформа товарных знаний на базе мультимодального GraphRAG

Домашнее задание 5. Курс OTUS “ИИ-архитектор”

Содержание

Контекст	1
1. Источники: что здесь на самом деле поток	2
Что от этого происходит с эталонным стеком	2
2. Схема потока данных	3
3. Зоны озера и граница между ETL и ELT	3
Где что происходит	5
4. Контроль качества	5
5. Выбор хранилищ	6
6. Датасет и его разбиение	7
7. Хранилище признаков и Training-Serving Skew	7
Решение: механизм тот же, инструмент другой	8
8. Происхождение данных и безопасность	8
9. Что мы теряем этими решениями	9
Итог	10

Контекст

Репозиторий проекта: https://github.com/SandalAndrey/otus_ai_architect

Заказчик - интернет-магазин коллекционных масштабных моделей: более 15 000 позиций в наличии, около 400 автомобильных марок, порядка 600 производителей.

До этой работы конвейер загрузки существовал в проекте как один прямоугольник на диаграмме уровня 2 с подписью “разбор документов, распознавание сканов,

извлечение сущностей”. Для оценки трудоёмкости этого хватало, для реализации нет: внутри не было ни зон хранения, ни оркестратора, ни проверок качества.

Реестр источников при этом был проработан раньше и подробно - приложение А к требованиям, одиннадцать источников с режимом “реальные или синтетические” и обоснованием по каждому. Настоящая работа достраивает над ним то, что происходит с данными дальше.

1. Источники: что здесь на самом деле поток

Разбор начинается с вопроса stream против batch, и ответ оказался неожиданным для схемы RAG: **поточковых источников нет ни одного.**

ID	Источник	Режим	Триггер	Свежесть
D-01	Карточки каталога сайта	Микропакет	Расписание, раз в час	NFR-4.2, 4 часа
D-02	Фотографии моделей	Пакет	Вместе с D-01	NFR-4.2
D-03	PDF-каталоги производителей	Пакет по событию	Появление файла	NFR-4.3, 24 часа
D-04	Прайс-листы поставщиков	Пакет по событию	Письмо или выгрузка	NFR-4.3, 24 часа
D-09	Остатки и сроки поставки	Чтение на лету, без ETL	Запрос пользователя	NFR-4.1, 15 минут
D-10	Обращения покупателей	Пакет, разово и ежемесячно	Выгрузка Заказчика	Не нормируется
D-11	Агрегаты нагрузки	Пакет, ежемесячно	Веб-аналитика	Не нормируется

Самое жёсткое требование - NFR-4.1, пятнадцать минут по остаткам - закрывается не конвейером, а его отсутствием. Остатки не попадают ни в озеро, ни в индекс: шлюз спрашивает их у учётной системы в момент формирования ответа. Решение принималось раньше и по другой причине (остаток меняется чаще, чем имеет смысл переиндексировать), но именно оно снимает потребность в потоковой обработке.

Кэш остатков не заводится сознательно: ключ кэша обязан включать набор разрешённых грифов пользователя, иначе ответ, построенный для закупщика, достанется покупателю.

Что от этого происходит с эталонным стеком

Типовой ответ на такую задачу выглядит как Kafka -> Spark -> S3 -> Pinecone. У нас от него остаётся один элемент из четырёх, и это стоит обосновать числами, а не предпочтением.

Элемент	Решение	Основание
Kafka	Не берём	Потоковых источников нет. Самое частое событие - обновление карточки, десятки в час. Брокер добавил бы узел, дежурство и точку отказа, не решая задачи
Spark	Не берём	Полный каталог - 15 000 позиций, единицы гигабайт вместе с документами. Помещается в память одного узла: Python и Polars
S3	Берём	Оригиналы документов обязаны храниться неизменными: без них ответ нельзя проследить до источника. On-premise это MinIO
Pinecone	Исключён	Закрытый контур (ADR-0009). Qdrant разворачивается у себя (ADR-0006)

Схема Lambda нам не подходит по той же причине: она решает расхождение потокового и пакетного представлений одних и тех же данных, а представление у нас одно. Карра предполагает, что всё является потоком событий. Наш случай - обычный пакетный конвейер плюс чтение свежих данных напрямую из источника в момент запроса.

2. Схема потока данных

Схема нарисована в Mermaid, а не в LikeC4, на котором ведётся вся остальная модель проекта. Это осознанное ослабление прежнего ограничения: C4 описывает элементы и связи, а поток данных описывает состояния одного набора и переходы между ними. Зона озера - не контейнер, проверка качества - не компонент. Обоснование и цена - ADR-0013.

3. Зоны озера и граница между ETL и ELT

Зона	Что лежит	Формат	Гриф по умолчанию
Bronze	Оригиналы как получены: прайс, PDF-каталог, выгрузка карточек	Исходный, байт в байт	Confidential
Silver	Нормализованные сущности и чанки с метаданными, до построения векторов	Parquet	По правилам FR-1.6
Gold	Витрины: узлы и связи графа, векторы чанков, датасет оценки	Neo4j, Qdrant, Parquet	По чанку

Граница между ELT и ETL проходит **по грифу, а не по слою**.

Для всего, что система анализирует, применяется ELT: файл сначала кладётся в Bronze без изменений. Причина в реестре рисков: разбор реальных прайсов плохого качества - главный риск конвейера (R-02), и переразбор того же файла новой версией парсера должен быть возможен без повторного запроса у поставщика.

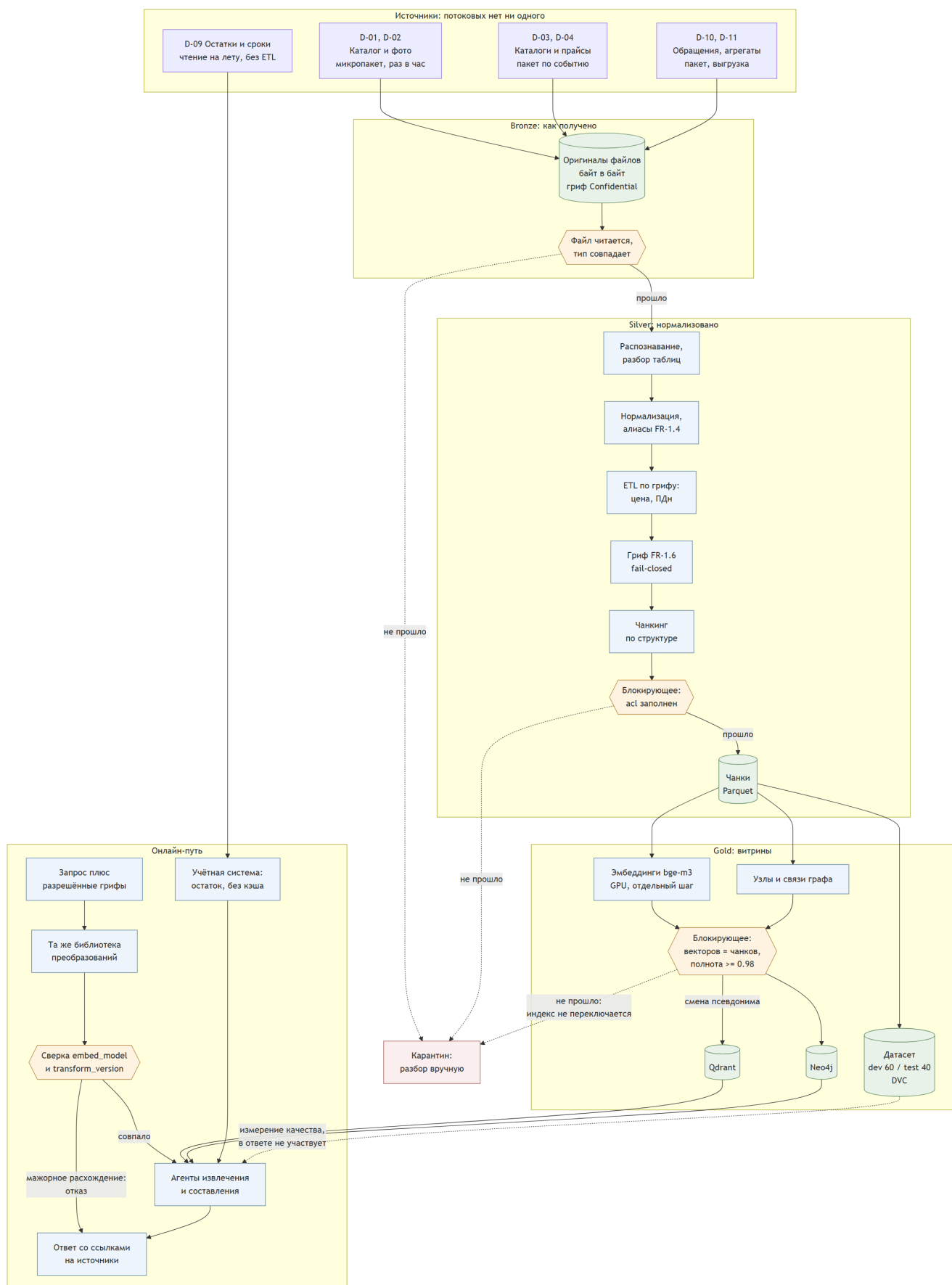


Рис. 1: Поток данных: источники, зоны озера, контроль качества, витрины, онлайн-путь. Исходник схемы - docs/diagrams/data-pipeline.mmd, картинка в полном разрешении

Для коммерческой тайны и персональных данных применяется ETL: подмена закупочной цены синтетическим значением и маскирование ПДн выполняются **до записи в Silver**, а не при выдаче. Озеро - такой же репозиторий со сроком хранения и правами; чем позже маскирование, тем больше мест, где оно может не сработать.

Отсюда следствие, которое надо назвать прямо: **Bronze - самое чувствительное место системы**. Там подлинные документы с реальными ценами и реквизитами, и защищает их не грифование чанков, а разграничение доступа к самой зоне. В проекте этот механизм пока описан одной строкой и приёмкой не проверяется. Это разрыв, он записан в последствиях ADR-0011 и закрывается на работе 1.5.

Где что происходит

- **Очистка и нормализация** - переход Bronze в Silver: распознавание сканов, разбор таблиц, приведение наименований, разрешение алиасов (FR-1.4), присвоение грифа (FR-1.6), подмена цены, маскирование ПДн.
- **Чанкинг** - там же, в конце перехода: границы чанка задаются структурой документа и не пересекают границу грифа.
- **Генерация эмбедингов** - отдельный шаг перехода Silver в Gold. Вынесен отдельно намеренно: это единственный шаг, требующий GPU, и единственный, который выполняется массово при смене модели. Смешав его с нормализацией, мы получили бы полный пересчёт нормализации при каждой смене модели эмбедингов.
- **Построение графа** - параллельный шаг того же перехода из тех же нормализованных сущностей.

4. Контроль качества

Проверки стоят на границах зон, а не в конце конвейера. Инструмент - Great Expectations.

Граница	Ожидание	При нарушении
Приём в Bronze	Файл открывается, непустой, тип соответствует расширению	Карантин, уведомление оператору
Bronze - Silver	Доля позиций, распознанных уверенно, не ниже 0.85	Партия в карантин, разбор вручную
Bronze - Silver	У каждого чанка заполнено поле asl	Блокирующее
Silver - Gold	Число векторов равно числу чанков партии	Блокирующее: индекс не переключается
Silver - Gold	Полнота индексации не ниже 0.98 (NFR-4.4)	Блокирующее
Ежесуточно	Профиль распределений: доля позиций без прототипа, доля сканов с низкой уверенностью, доля чанков каждого грифа	Предупреждение при отклонении более 20 процентов

Блокирующих ожиданий три, и все три про безопасность или полноту, а не про удобство. Правило fail-closed действует и здесь: чанк без определённого грифа **не индексируется вовсе**, а не индексируется как Confidential. Разница существенная: во втором случае документ тихо становится недоступен всем, и обнаруживается это жалобой пользователя.

Профилирование нужно отдельно от ожиданий, потому что ловит другое. Ожидание отвечает на вопрос “данные корректны”, профиль - на вопрос “источник не изменился”. Поставщик, поменявший вёрстку прайса, пройдёт все ожидания и тихо уронит полноту извлечения.

5. Выбор хранилищ

Назначение	Технология	Почему эта
Озеро, три зоны	MinIO	On-premise, стандартный протокол S3, уже есть на диаграмме уровня 2
Табличный формат Silver и Gold	Parquet	Схема внутри файла. Iceberg и Delta решают конкурентную запись и эволюцию схемы на масштабе, которого нет: один писатель, гигабайты
Граф знаний	Neo4j	ADR-0006. Статус записи “Предложено” до ответа на вопрос об импортозамещении
Векторное хранилище	Qdrant	Там же. Определяющее требование - фильтр по грифу одновременно с поиском соседей, а не после
Состояние конвейера, аудит	PostgreSQL	Уже в архитектуре как служебная база
Оркестрация	Dagster	Граф активов даёт происхождение данных как побочный продукт, а не как отдельный документ
Контроль качества	Great Expectations	Ожидания хранятся кодом в репозитории и проходят ревью наравне с остальным
Версионирование датасетов	DVC	Датасет остаётся в объектном хранилище, в репозиторий попадает указатель
Хранилище признаков	He	Раздел 7
	MinIO	заводится

Отдельного аналитического хранилища нет. Аналитические вопросы есть - полнота индексации, распределение грифов, качество распознавания, - но отвечаются запросом к Parquet напрямую. ClickHouse добавил бы узел ради задачи, которая решается без него.

Про оркестратор стоит сказать честную часть: **Airflow зрелее и распространёнее, найм под него проще**. Dagster выбран за одно свойство - он оркестрирует активы, а не задачи, и зависимость “векторы зависят от чанков” выражается в нём связью данных, а не порядком запуска. Без хранилища признаков это единственный источник происхождения данных, который не надо вести руками. При наличии в команде опыта эксплуатации Airflow решение стоит пересмотреть.

6. Датасет и его разбиение

Обучения в проекте нет: модели берутся готовыми и не дообучаются. Поэтому обучающей выборки не существует, и это следствие ограничения, а не упущение. Датасет нужен для измерения.

Выборка	Доля	Назначение
dev	60 %	Подбор параметров извлечения: глубина кандидатов, порог слияния, размер контекста
test	40 %	Приёмка по NFR-2. Вскрывается один раз перед Decision Gate

Разбиение стратифицировано по двум признакам: классу вопроса (простой или многошаговый) и популярности марки. Второе - прямое следствие требования о справедливости подбора аналогов: если редкие марки попадут преимущественно в одну выборку, метрика станет недостоверной.

Правило, без которого разбиение бессмысленно: **параметры подбираются только на dev**. Подбор на test даёт завышенную оценку, которая разваливается на продуктиве, и заметить это на приёмке нельзя.

Версионирование - DVC, тег на каждую версию. В отчёт по метрикам записывается хеш версии датасета и версия конвейера извлечения: без этого сравнение двух прогонов ничего не значит, неизвестно, изменилась система или изменился датасет.

7. Хранилище признаков и Training-Serving Skew

Это самый интересный пункт разбора, потому что первый напрашивающийся ответ неверен.

Хранилище признаков решает конкретную задачу: не дать разойтись признакам, вычисленным при обучении, и признакам, вычисленным при работе модели. Если код нормализации существует в двух экземплярах - в пакетном конвейере и в онлайн-сервисе, - они расходятся, модель получает не то, на чём училась, и качество падает без единой ошибки в журналах.

Напрашивается сказать, что нас это не касается: моделей мы не обучаем. Вывод неверный. **Обучения нет, а двух путей вычисления одного представления - два**, и дефект той же природы возможен.

Текст проходит преобразования дважды. При индексации: наименование нормализуется, артикул выделяется, поля собираются в текст чанка, из него строится плотный и разреженный вектор. При запросе: вопрос пользователя и его переформулировки проходят те же преобразования. Поиск осмыслен ровно настолько, насколько эти два преобразования совпадают.

Разойтись они могут четырьмя способами, и все четыре тихие:

- **Версия модели эмбедингов.** Обновили модель в онлайн-пути, индекс не переиндексировали. Размерность совпадает, поиск работает, выдача деградирует.

- **Нормализация текста.** Регистр, смешанное написание марок кириллицей и латиницей, дефисы в артикулах. Бьёт по классу запросов, а не по всем.
- **Состав текста чанка.** При индексации в текст добавлялось наименование производителя, при запросе аналогичного обогащения нет - плотный поиск смещается.
- **Разреженная часть.** Разная токенизация ломает поиск по артикулам, то есть самые точные запросы.

Общее у всех: **ни один не вызывает ошибки.** Система отвечает, пользователь получает правдоподобный текст, метрики доступности зелёные.

Решение: механизм тот же, инструмент другой

Хранилище признаков не заводится, потому что признаков для обучения не существует. Механизм, ради которого его берут, воспроизводится тремя способами.

1. **Один код преобразований.** Нормализация наименований, выделение артикулов и сборка текста чанка вынесены в общую библиотеку, вызываемую и конвейером загрузки, и агентом векторного поиска. Второй реализации нет - это и есть главный механизм хранилища признаков. Требование к библиотеке: чистые функции над текстом, без обращений к базам и сети, иначе её нельзя вызвать из обоих контекстов.
2. **Версия в полезной нагрузке.** Каждый чанк несёт `embed_model` и `transform_version`. При запросе версии сверяются. Расхождение по мажорной версии - отказ с явной ошибкой, а не тихая выдача: отвечать на индексе неизвестной совместимости хуже, чем не ответить.
3. **Атомарное переключение индекса.** Новая коллекция строится рядом со старой, переключение - сменой псевдонима, откат - обратной сменой. Без этого первые два механизма бессмысленны: во время длительной переиндексации индекс содержал бы обе версии.

Отдельно: изменение грифа документа инвалидирует его чанки немедленно, не дожидаясь переиндексации. Обратный порядок оставляет окно, в котором закрытый документ доступен, и приёмка это окно не покажет.

Что мы теряем, отказавшись от инструмента, - в разделе 9.

8. Происхождение данных и безопасность

Хранилища признаков нет, поэтому происхождение строится двумя механизмами, и оба обязательны.

Сверху, между активами. Dagster описывает конвейер как граф активов. Зависимость “векторы зависят от чанков, чанки зависят от разобранного документа” объявляется в коде и отображается сама. Вопрос “что сломается, если переразобрать этот прайс” отвечает инструмент, а не память инженера.

Снизу, по записи. Каждый чанк несёт `doc_id`, `source_type`, `page`, `content_hash` и добавленное здесь `ingest_run_id`. Этого достаточно, чтобы от предъявленного пользователю ответа дойти до конкретного файла в Bronze и до прогона конвейера, который его обработал.

Вторая цепочка нужна не для отладки: требование обязывает сопровождать каждое утверждение ссылкой на источник, а журнал хранится двенадцать месяцев. Ответ, для которого через год нельзя предъявить исходный документ, требованию не соответствует, даже если он верен.

Безопасность данных по слоям:

Мера	Где
Подмена закупочной цены	Переход Bronze в Silver, до записи
Маскирование ПДн	Там же, плюс guardrails на входе и выходе онлайн-пути
Синтетические договоры и ПДн	Загружаются как подготовленный набор, конвейером не генерируются
Гриф на чанке и узле	FR-1.6, fail-closed
Разграничение доступа к зонам	Отдельные сервисные учётные записи; онлайн-путь доступа к Bronze не имеет вовсе
Данные вне репозитория	Коммитятся генераторы и схемы, не наборы

Последнее существенно: агенты читают только Gold, где гриф проставлен, а цена подменена. Это отделяет вопрос “правильно ли работает фильтрация” от вопроса “что лежит в оригинале документа”, и первый становится проверяемым.

9. Что мы теряем этими решениями

Раздел отдельный, потому что без него набор решений выглядит бесплатным.

- **Слой данных не масштабируется без переделки.** Отсутствие брокера и распределённой обработки станет ограничением при росте объёма на порядок или появлении потоковых источников: слой придётся строить заново, а не расширять. Мы выбрали дешёвое решение для известного объёма вместо дорогого для предполагаемого.
- **Объём не подтверждён.** Весь выбор опирается на порядок величины из наблюдения за каталогом, а не на ответ Заказчика (вопрос Q-12). Если архив документов исчисляется терабайтами, ADR-0011 пересматривается целиком.
- **Консистентность держится на соглашениях, а не на инструменте.** Ничто технически не мешает написать вторую нормализацию прямо в агенте. Хранилище признаков сделало бы это дороже. Компенсация слабая: ревью и тест, прогоняющий одну строку через оба пути и сравнивающий векторы.
- **Атомарное переключение требует двойного объёма индекса** на время переиндексации. Для Qdrant это память, и на целевом железе её надо учитывать сверх прежнего расчёта.
- **Отказ при расхождении версий виден пользователю.** Ошибка развёртывания превращается в недоступность, а не в тихую деградацию. Считаем такой размен правильным, но это размен.
- **Dagster в команде незнаком.** Меняем доступность компетенции на встроенное происхождение данных.

- **В проекте стало две нотации диаграмм.** Ограничение ADR-0003 ослаблено, и расхождение схем между собой ничем не проверяется: сверка четырёх общих элементов ручная.

Итог

Что сделано	Где
Конвейер данных: источники, зоны, качество, хранилища, датасет, происхождение	docs/13-data-pipeline.md
Схема потока данных	docs/diagrams/data-pipeline.mmd
ADR-0011: озеро, Dagster, Great Expectations, отказ от Kafka и Spark	docs/adr/0011-...
ADR-0012: отказ от хранилища признаков, консистентность версией	docs/adr/0012-...
ADR-0013: вторая нотация для потока данных	docs/adr/0013-...

Главный вывод работы не про инструменты. Разбор начинался с ожидания, что задача сведётся к выбору между Kafka и чем-то попроще, а свелась к другому: **в системе, которая ничего не обучает, дефект вида training-serving skew всё равно возможен**, потому что для его появления достаточно двух путей вычисления одного представления, а не наличия обучения. Хранилище признаков нам не нужно как инструмент, но нужно как понятая идея - и три механизма, которые из неё выведены, в проекте появились.

Второй вывод скучнее и полезнее: эталонный стек не подошёл ни в одном из трёх пунктов, где у нас есть числа. Не потому, что он плох, а потому, что он рассчитан на объём, которого у нас нет. Взять его целиком было бы проще для защиты и дороже в эксплуатации.